ASSIGNMENT NO. 6

INTER PROCESS COMMUNICATION USING FIFO

AIM : Full duplex communication between two independent processes. First process accepts sentences and writes on one pipe to be read by second process and second process counts number of characters, number of words and number of lines in accepted sentences, writes this output in a text file and writes the contents of the file on second pipe to be read by first process and displays on standard output.

OBJECTIVES : To study

- FIFO
- FIFO Operations
- Use of FIFO for inter process communication

THEORY :

FIFOs

A FIFO (First In First Out) is a one-way flow of data. FIFOs have a name, so unrelated processes can share the FIFO. FIFO is a named pipe. Any process can open or close the FIFO. FIFOs are also called named pipes.

Properties:

1. After a FIFO is created, it can be opened for read or write.
2. Normally, opening a FIFO for read or write, it blocks until another process opens it for write or read.
2. A read gets as much data as it requests or as much data as the FIFO has, whichever is less.
3. A write to a FIFO is atomic, as long as the write does not exceed the capacity of the FIFO.
4. FIFO must be opened by two processes; one opens it as reader on one end, the other opens it as sender on the other end. The first/earlier opener has to wait until the second/later opener to come. This is somewhat like a hand-shaking.

Creating a FIFO

A FIFO is created by the mkfifo function. Specify the path to the FIFO on the command line. For example, create a FIFO in /tmp/fifo by invoking this:

```
#include <sys/types.h>
#include <sys/stat.h>
        Int mkfifo(const char *pathname, mode_t mode);
```

pathname: a UNIX pathname (path and filename). The name of the FIFO

mode: the file permission bits. It specifies the pipe's owner, group, and world permissions, and a pipe must have a reader and a writer, the permissions must include both read and write permissions.

If the pipe cannot be created (for instance, if a file with that name already exists), mkfifo returns –1.

FIFO can also be created by the mknod system call,

e.g., mknod("fifo1", S_IFIFO|0666, 0) is same as mkfifo("fifo1", 0666).

Accessing a FIFO

Access a FIFO just like an ordinary file. To communicate through a FIFO, one program must open it for writing, and another program must open it for reading. Either low-level I/O functions like open, write, read, close or C library I/O functions (fopen, fprintf, fscanf, fclose, and soon) may be used.

For example, to write a buffer of data to a FIFO using low-level I/O routines, you could use this code:

```
Intfd = open (fifo_path, O_WRONLY);
write (fd, data, data_length);
close (fd);
```

To read a string from the FIFO using C library I/O functions, you could use this code:

```
FILE* fifo = fopen (fifo_path, "r");
fscanf (fifo, "%s", buffer);
fclose (fifo);
```

A FIFO can have multiple readers or multiple writers. Bytes from each writer are written atomically up to a maximum size of PIPE_BUF (4KB on Linux). Chunks from simultaneous writers can be interleaved. Similar rules apply to simultaneous reads.

Close: To close an open FIFO, use close().

Unlink : To delete a created FIFO, use unlink().

Program 6A1:

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<string.h>
#define max_buf 100

int main()
{
 char *myfifo1="myfifo1",*myfifo2="myfifo2",arr[50];
 mkfifo(myfifo1,0666);
 int i,words,lines;
 int fd;
 fgets(arr,50,stdin);
 fd=open(myfifo1,O_WRONLY);
 write(fd,arr,strlen(arr)+2);
 close(fd);
 unlink(myfifo1);

 int fd1;
 fd1=open(myfifo2,O_RDONLY);
 read(fd1,&i,sizeof(i));
 printf("Total characters: %d\n",i);
 read(fd1,&words,sizeof(words));
 printf("Total word: %d\n",words);
 read(fd1,&lines,sizeof(lines));
 printf("Total Lines: %d\n",lines);
 close(fd1);
}
```

Program 6A2:

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#define max_buf 100

int main()
{
 char *myfifo1="myfifo1",*myfifo2="myfifo2";
 char buf[50];
 FILE *fp;
 int i=0,words=0,lines=0;
 mkfifo(myfifo2,0777);
 int fd,fd1;
 fd=open(myfifo1,O_RDWR);
 read(fd,buf,max_buf);
 printf("\nMessage received is: %s",buf);
 while(buf[i]!='\0')
 {
  while(buf[i]==' ')
  {
   words++,i++;
  }
  if(buf[i]=='.'||buf[i]=='?'||buf[i]=='!')
  {
   lines++,i++;
  }
  i++;
 }
 printf("\n Total no. of characters:%d",i);
  fp=fopen("abc.txt","w+");
  fprintf(fp,"Total characters=%d",i);

  printf("\n Total no. of words:%d",words);
  fp=fopen("abc.txt","w+");
  fprintf(fp,"Total characters=%d",words);

  printf("\n Total no. of characters:%d",lines);
  fp=fopen("abc.txt","w+");
  fprintf(fp,"Total no. of lines=%d",lines);
  fclose(fp);
  unlink(myfifo1);
  fd1=open(myfifo2,O_RDWR);
  write(fd1,&i,sizeof(i));
  write(fd1,&words,sizeof(words));
  write(fd1,&lines,sizeof(lines));
  close(fd1);
 return 0;}
```

CONCLUSION: Thus, we studied inter process communication using FIFOs.